

Laporan Praktikum Kontrol Cerdas Week 3

Nama : Wahyu Agustina
NIM : 224308046
Kelas : TKA-6B
Akun Github : <https://github.com/Wahyuagustina24>
Student Lab Assistant : Mba Salma Halizah

1. Judul Percobaan: *Image Classification with CNN*

2. Tujuan Percobaan:

Tujuan dari percobaan *Image Classification with CNN* adalah :

- Memahami konsep dasar *Deep Learning* dalam sistem kendali.
- Mengimplementasikan *Convolutional Neural Network* (CNN) untuk klasifikasi objek.
- Menggunakan *TensorFlow* dan Keras untuk membangun model *Deep Learning*.
- Mengintegrasikan model CNN dengan *Computer Vision* untuk deteksi objek secara *real-time*.
- Menggunakan dataset dari Kaggle untuk pelatihan model.
- Mengembangkan *mode Night Vision* untuk deteksi objek dalam kondisi pencahayaan rendah.

3. Landasan Teori:

Kecerdasan buatan atau *Artificial Intelligence* (AI) adalah sistem yang mempelajari bagaimana membuat komputer dapat berpikir, belajar, dan bertindak seperti manusia (Dosari & Abouellail, 2023). *Intelligent Control* adalah metode pengendalian sistem yang menggunakan AI, *Machine Learning*, dan *Deep Learning* untuk meningkatkan performa dan efisiensi. Contoh implementasi *Intelligent Control* yaitu pada robotika (kontrol gerak robot berbasis AI), otomotif (*autonomous driving system*), industri (prediksi dan optimasi proses manufaktur), dan medis (AI untuk kontrol peralatan kesehatan) (Marsella dkk., 2023).

Machine Learning adalah pendekatan *Artificial Intelligence* yang berfokus pada pembuatan mesin (robot) yang dapat belajar tanpa diprogram secara eksplisit (detail). *Machine Learning* memungkinkan sistem kendali untuk belajar dari data dan meningkatkan performanya seiring waktu. *Deep Learning* adalah salah satu cabang ilmu dari *Machine Learning* yang memanfaatkan jaringan syaraf tiruan untuk implementasi permasalahan dengan dataset besar. *Deep Learning* dapat memberikan hasil yang lebih akurat karena proses ini seperti meniru cara kerja otak manusia. Salah satu metode dalam *Deep Learning* yang mampu memberikan hasil signifikan mengenai pengenalan objek gambar adalah CNN. CNN adalah salah satu jenis *neural network* yang biasa digunakan pada data image dan dapat mendeteksi serta mengenali objek sebuah *image* (“[No title found],” t.t.).

Convolutional Neural Network (CNN) adalah arsitektur *Deep Learning* yang sering digunakan untuk mengatasi masalah klasifikasi gambar. Selain itu, penggunaan teknik *pre-processing* pada data dapat membantu meningkatkan performa model dengan memperkaya variasi data. Metode *Convolutional Neural Network* sangat populer di kalangan *Deep Learning*, karena CNN mengekstrak fitur dari input yang berupa gambar lalu mengubah dimensi gambar tersebut menjadi lebih kecil tanpa merubah karakteristik gambar tersebut (Azmi dkk., 2023). *Convolutional Neural Network* (CNN) terdiri dari neurons yang memiliki bobot dan bias. Setiap neurons menerima inputan dan diteruskan dengan melakukan perkalian titik pada setiap neuron tersebut. CNN merupakan metode yang efektif untuk mengklasifikasikan, mengidentifikasi, dan mengenali pola di dalam citra. CNN mampu memahami detail gambar dengan lebih baik karena memiliki arsitektur yang sesuai dengan cara otak manusia memproses informasi visual. Data yang digunakan pada CNN adalah data dua dimensi, seperti citra atau suara, dan menggunakan operasi konvolusi dalam matriks serta bobot yang berbentuk empat dimensi yang merupakan sekumpulan kernel konvolusi (Anhar & Putra, 2023).

Software yang digunakan dalam percobaan ini berupa Python dan Open CV. OpenCV (*Open Source Computer Vision Library*) adalah perpustakaan perangkat lunak yang dirancang khusus untuk pengolahan citra dan visi

komputer. Dikembangkan oleh Intel dan sekarang bersifat *open-source*, OpenCV menyediakan berbagai alat dan fungsi yang memudahkan pengembang untuk membuat aplikasi berbasis pengolahan citra dan visi komputer. Python, sebagai salah satu bahasa pemrograman yang didukung oleh OpenCV, menjadi pilihan populer karena sintaksnya yang mudah dipahami dan kemampuannya untuk integrasi dengan berbagai pustaka lain, seperti NumPy, SciPy, dan scikit-learn. Dengan OpenCV Python, pengembang dapat mengakses berbagai algoritma canggih untuk tugas-tugas seperti deteksi objek, pelacakan, pengenalan wajah, dan segmentasi citra. Sehingga bahasa pemrograman Python yang diprogram memiliki keahlian yaitu memberikan hasil keluaran sesuai input dari dataset, metode dan algoritma yang digunakan (Zebua & Rosyani, 2024).

Support Vector Machine adalah model ML multifungsi yang dapat digunakan untuk menyelesaikan permasalahan klasifikasi, regresi, dan pendeteksian *outlier*. *Support Vector Machine* merupakan salah satu metode klasifikasi dengan menggunakan metode *Machine Learning (supervised learning)* yang memprediksi kelas berdasarkan pola dari hasil proses *training* (Aisah dkk., 2023).

4. Analisis dan Diskusi:

- Analisis

Pada praktikum minggu ketiga ini percobaan yang dilakukan adalah membuat klasifikasi gambar yang terdeteksi dengan model *Deep Learning* menggunakan algoritma *Convolutional Neural Network* (CNN). Tahap pertama yang dilakukan adalah Setup Environment dan Dataset. Selanjutnya, muat dataset dan lakukan *pre-processing* untuk melatih model CNN atau klasifikasi gambar menggunakan Keras atau TensorFlow. Kemudian, kode **ImageDataGenerator** digunakan untuk melakukan augmentasi data pada dataset pelatihan. Augmentasi data adalah teknik untuk memperbanyak variasi data pelatihan dengan melakukan transformasi seperti rotasi, *zoom*, dan *flip* horizontal. Hal ini membantu model menjadi lebih *robust* dan mencegah *overfitting*.

Tahap kedua yang dilakukan adalah membuat dan melatih model CNN untuk klasifikasi objek. Pertama, mendefinisikan model CNN menggunakan Keras Sequential API. Kode **model = Sequential** Ini membuat model sequential, yang berarti lapisan-lapisan dalam model akan dieksekusi secara berurutan. Selanjutnya, mengkompilasi model CNN yang telah kita definisikan sebelumnya menggunakan kode **model.compile**. Kompilasi adalah langkah penting sebelum kita dapat melatih model. Lalu, kode **model.fit** adalah fungsi yang digunakan untuk melatih model. Ini adalah langkah kunci dalam proses pembelajaran mesin, di mana model belajar untuk memetakan input ke output yang diinginkan. Dan terakhir, model yang terlatih disimpan ke file **cnn_model.h5**.

Tahap ketiga yang dilakukan adalah mengintegrasikan dengan OpenCV untuk prediksi *Real-time* dan *Night Vision*. Dengan menggunakan kamera inisialisasi program untuk mendeteksi objek berdasarkan model CNN yang telah dibuat dengan tambahan fitur Night Vision. Pertama, load model yang telah dilatih menggunakan kode **model = load_model**. Kemudian, load label kelas menggunakan kode **class_labels = list** yang bertujuan untuk mengonversi output numerik dari model CNN (biasanya berupa indeks atau vektor probabilitas) menjadi label kelas yang dapat dibaca manusia.

Selanjutnya, data fitur (nilai RGB) dinormalisasi menggunakan kode **scaler = StandardScaler()**. Normalisasi ini penting untuk memastikan bahwa semua fitur memiliki skala yang sama sehingga dapat meningkatkan kinerja model KNN. Lalu, dilakukan pemisahan dataset menjadi dua bagian yaitu, set pelatihan (*training set*) dan set pengujian (*testing set*) yang digunakan untuk mengevaluasi kinerja model pada data yang tidak dilihat selama pelatihan. Selanjutnya yang dilakukan adalah menginisialisasi kamera menggunakan **cap = cv2.VideoCapture** dengan kamera bawaan PC/Laptop atau dengan *webcam*. Kemudian, menginisialisasi mode *Night Vision* dengan konversi ke skala abu-abu menggunakan kode **night_vision = cv2.cvtColor**. Konversi ke skala abu-

abu adalah teknik sederhana namun efektif untuk meningkatkan visibilitas gambar dalam kondisi cahaya redup, yang sering digunakan dalam aplikasi penglihatan malam. Kode **img = cv2.resize** merupakan bagian dari pra-pemrosesan gambar yang sangat penting dalam aplikasi deteksi *objek real-time* menggunakan model CNN. Fungsi **cv2.resize** dari pustaka OpenCV digunakan untuk mengubah ukuran gambar yang diambil dari kamera (frame) menjadi dimensi yang sesuai dengan input model CNN. Kemudian, memprediksi kelas menggunakan kode **model.predict(img)** untuk mengambil gambar yang telah diproses sebelumnya (img) sebagai input dan menghasilkan prediksi. Prediksi ini biasanya berupa array probabilitas, di mana setiap elemen array mewakili probabilitas bahwa gambar tersebut termasuk dalam kelas tertentu. Lalu, mengambil label kelas yang sesuai dari daftar **class_labels**. Dimana **class_labels** adalah daftar yang berisi nama-nama kelas yang sesuai dengan indeks prediksi. Dengan demikian, label akan berisi nama kelas yang diprediksi oleh model untuk gambar input. Proses ini memungkinkan kita untuk mengubah prediksi numerik dari model menjadi label kelas yang dapat dibaca manusia. Terakhir, menampilkan hasil menggunakan kode **cv2.putText** untuk menambahkan teks ke dalam frame video, kode **cv2.imshow('Frame', frame)** menampilkan frame video asli yang telah ditambahkan teks, dengan judul jendela 'Frame', dan kode **cv2.imshow('Night Vision', night_vision)** untuk menampilkan frame video yang telah diproses untuk fitur penglihatan malam, dengan judul jendela 'Night Vision'. Kedua fungsi **cv2.imshow** ini digunakan untuk melihat hasil pemrosesan video secara *real-time*.

Kemudian, dilakukan modifikasi program dengan menambahkan program **LUX_CALIBRATION_FA** untuk menampilkan faktor kalibrasi untuk perhitungan nilai lux. Dalam konteks ini, faktor kalibrasi digunakan untuk menyesuaikan hasil perhitungan intensitas cahaya agar lebih akurat. Kemudian, mengkategorikan beberapa pencahayaan seperti “gelap”, “redup”, “sedang”, “cukup terang”, dan “terang” untuk mengubah nilai numerik intensitas cahaya menjadi deskripsi kualitatif, sehingga lebih

mudah bagi pengguna untuk memahami tingkat pencahayaan. Setelah itu, dengan menggunakan mode *Night Vision* diperhitungkan juga nilai lux nya dengan menggunakan kode **lux_value = luminance * LUX_CALIBRATION_FACTOR**. Kemudian, dimodifikasi juga dengan menambahkan waktu *real-time* dari sistem dan memformatnya menjadi *string* yang menampilkan jam, menit, dan detik menggunakan kode **current_time = datetime.datetime.now().strftime('%H:%M:%S')**. Selain itu, yang terakhir menyimpan data nilai lux dan waktu ke dalam daftar (list) untuk pembuatan grafik. Tujuannya adalah untuk menghindari duplikasi data yang akan membuat grafik menjadi terlalu padat dan sulit dibaca, terutama jika data waktu memiliki resolusi detik dengan menggunakan kode **if current_time not in seen_timestamps: | timestamps.append(current_time) | lux_values.append(lux_value) | seen_timestamps.add(current_time)**.

Perbedaan hasil kondisi kelas dan lux dari percobaan yang telah dilakukan dapat disebabkan akibat sejumlah faktor, terutama yang berkaitan dengan kualitas model pengenalan objek dan akurasi pengukuran cahaya. Untuk perbedaan pengenalan kelas dapat disebabkan oleh beberapa faktor seperti akurasi model pembelajaran mesin, kondisi pencahayaan, variasi objek, dan kualitas kamera. Model yang kurang terlatih atau data pelatihan yang tidak representatif dapat menghasilkan prediksi kelas yang salah, sementara perubahan pencahayaan atau variasi objek yang tinggi dapat mengganggu deteksi fitur.

Sedangkan, pengukuran lux sangat bergantung pada metode penghitungan, kondisi lingkungan seperti debu dan kelembaban, serta kualitas kamera. Penghitungan lux yang didasarkan pada rata-rata nilai piksel frame mungkin tidak seakurat sensor lux meter khusus, dan perbedaan cara penghitungan dapat memberikan hasil yang berbeda.

- Diskusi

Metode klasifikasi objek yang terdeteksi dengan model *Deep Learning* menggunakan *Convolutional Neural Network* (CNN) dengan penambahan fitur *Night Vision*, menampilkan lux pencahayaan dan

menambahkan waktu *real-time* serta grafik ini memiliki kelebihan dan kekurangan. Kelebihannya antara lain fitur *Night Vision* memungkinkan sistem untuk tetap berfungsi efektif dalam kondisi pencahayaan yang minim, penambahan deteksi lux memberikan informasi kuantitatif tentang intensitas cahaya, yang berguna untuk analisis lingkungan, deteksi kelas memungkinkan identifikasi objek secara otomatis, penambahan waktu *real-time* memungkinkan pemantauan kejadian secara kronologi, dan grafik memvisualisasikan data lux agar pola lebih mudah dipahami. Sedangkan kekurangannya antara lain data pelatihan yang tidak memadai atau tidak representatif dapat menghasilkan deteksi objek yang buruk, pengukuran lux memerlukan kalibrasi yang cermat untuk akurasi optimal, dan pembuatan grafik membutuhkan sumber daya komputasi yang cukup.

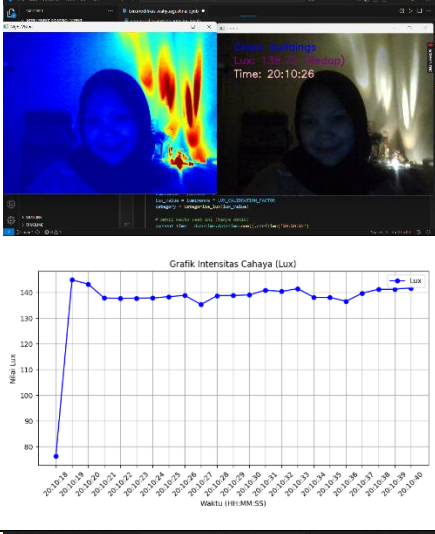
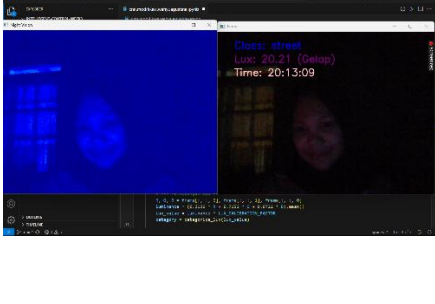
5. *Assignment:*

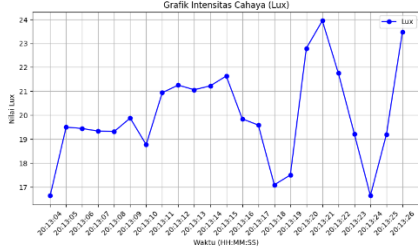
Dalam praktikum yang telah dilakukan mengenai modifikasi program dengan model *Deep Learning* menggunakan *Convolutional Neural Network* (CNN) dengan penambahan fitur *Night Vision*, menampilkan lux pencahayaan, menambahkan waktu *real-time* serta grafik. Tahapan percobaan yang dilakukan meliputi pertama setup environment dan dataset (memuat dataset dan melakukan *pre-processing*), kedua pembuatan dan pelatihan model CNN (mendefinisikan, mengkompilasi, dan melatih model CNN), ketiga mengintegrasikan dengan OpenCV (menggunakan kamera untuk deteksi objek *real-time* dengan fitur *Night Vision*) dan keempat memodifikasi program (menambahkan faktor kalibrasi lux, kategorisasi pencahayaan, perhitungan nilai lux, waktu *real-time*, dan penyimpanan data untuk grafik).

Dari hasil percobaan diperoleh perbedaan kondisi kelas dan lux yang dipengaruhi oleh kualitas model, kondisi pencahayaan, variasi objek, kualitas kamera, metode penghitungan lux, dan kondisi lingkungan. Dengan menggunakan model *Convolutional Neural Network* (CNN) dengan penambahan fitur *Night Vision*, menampilkan lux pencahayaan, menambahkan waktu *real-time* serta grafik, sistem memiliki kelebihan dalam akurasi deteksi objek, adaptasi terhadap cahaya rendah, informasi pencahayaan yang lengkap, pemantauan *real-time*, visualisasi data yang mudah dipahami, dan potensi

otomatisasi. Namun, juga memiliki kekurangan meliputi kebutuhan komputasi yang tinggi, ketergantungan pada data pelatihan, kalibrasi yang rumit, kompleksitas implementasi, potensi kesalahan, dan kebutuhan sumber daya yang besar.

6. Data dan Output Hasil Pengamatan:

No	Kondisi	Hasil Percobaan	Output
1.	Didalam kamar dengan kondisi lampu utama menyala		Dari hasil percobaan dengan kondisi didalam kamar dan lampu utama menyala diperoleh hasil deteksi: Class: Building Lux: 324.94 (Cukup Terang)
2.	Didalam kamar dengan kondisi lampu utama mati dan lampu tidur menyala		Dari hasil percobaan dengan kondisi didalam kamar dan lampu utama mati sedangkan lampu tidur menyala diperoleh hasil deteksi: Class: Building Lux: 138.72 (Redup)
3.	Didalam kamar dengan kondisi lampu utama mati		Dari hasil percobaan dengan kondisi didalam kamar dan lampu utama mati diperoleh hasil deteksi: Class: street

			Lux: 20.21 (Gelap)
4.	Di teras rumah dengan kondisi setelah hujan		Dari hasil percobaan dengan kondisi didalam kamar dan lampu utama menyala diperoleh hasil deteksi: Class: Forest Lux: 168.33 (Sedang)

7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat ditarik kesimpulan yaitu:

- Pada praktikum percobaan ini berhasil mengimplementasikan model *Convolutional Neural Network* (CNN) untuk mengklasifikasi objek dengan integrasi OpenCV dan fitur *Night Vision* untuk meningkatkan deteksi objek dalam kondisi pencahayaan rendah.
- Penambahan deteksi lux memberikan informasi kuantitatif tentang intensitas cahaya dalam berbagai kondisi lingkungan.
- Modifikasi seperti menampilkan waktu *real-time* dan pembuatan grafik, memungkinkan analisis lebih lanjut terhadap data pencahayaan.
- Perbedaan hasil klasifikasi dan pengukuran lux dipengaruhi oleh kualitas model, kondisi pencahayaan, variasi objek, dan kualitas kamera.

8. Saran:

Untuk meningkatkan performa sistem, sebaiknya dilakukan optimalisasi model CNN dengan dataset yang lebih besar dan lebih bervariasi guna meningkatkan akurasi klasifikasi. Selain itu, penggunaan sensor cahaya tambahan dapat membantu dalam mengukur lux secara lebih akurat dibandingkan hanya mengandalkan nilai piksel dari kamera. Efisiensi pemrosesan juga perlu ditingkatkan dengan menerapkan teknik kompresi model atau optimasi kode pada OpenCV dan TensorFlow agar performa sistem lebih cepat dan ringan. Terakhir, uji coba dalam berbagai lingkungan dengan pencahayaan yang lebih kompleks disarankan untuk meningkatkan ketahanan sistem dalam kondisi nyata.

9. Daftar Pustaka:

- Aisah, I. S., Irawan, B., & Suprpti, T. (2023). *ALGORITMA SUPPORT VECTOR MACHINE (SVM) UNTUK ANALISIS SENTIMEN ULASAN APLIKASI AL QUR'AN DIGITAL*. 7(6).
- Anhar, A., & Putra, R. A. (2023). Perancangan dan Implementasi Self-Checkout System pada Toko Ritel menggunakan Convolutional Neural Network (CNN). *ELKOMIKA: Jurnal Teknik Energi Elektrik, Teknik Telekomunikasi, & Teknik Elektronika*, 11(2), 466. <https://doi.org/10.26760/elkomika.v11i2.466>
- Azmi, K., Defit, S., & Sumijan, S. (2023). Implementasi Convolutional Neural Network (CNN) Untuk Klasifikasi Batik Tanah Liat Sumatera Barat. *JURNAL UNITEK*, 16(1), 28–40. <https://doi.org/10.52072/unitek.v16i1.504>
- Dosari, F. H. M. A., & Abouellail, S. I. A. D. (2023). Artificial Intelligence (AI) Techniques for Intelligent Control Systems in Mechanical Engineering. *American Journal of Smart Technology and Solutions*, 2(2), 55–64. <https://doi.org/10.54536/ajsts.v2i2.2188>
- Marsella, M., Wijaya, C. S., Wijaya, I., Shidqi, M. T., & Novita, D. (2023). ANALISIS IMPLEMENTASI ARTIFICIAL INTELLIGENCE UNTUK BISNIS: SYSTEMATIC LITERATURE REVIEW. *DEVICE : JOURNAL OF INFORMATION SYSTEM, COMPUTER SCIENCE*

AND INFORMATION TECHNOLOGY, 4(2), 133–145.
<https://doi.org/10.46576/device.v4i2.4037>

[No title found]. (t.t.). *Pattimura Proceeding: Conference of Science and Technology*.

Zebua, E. T. P., & Rosyani, P. (2024). *Perancangan Deteksi Objek Kendaraan Bermotor Berbasis OpenCV Python menggunakan Metode HOG-SVM untuk Analisis Lalu Lintas Cerdas*. 2(1).